

Assignment 2

A Simple CUDA Program: Host and Device Code

Name: Arnav Singhal

MIS: 612203012

Div: 1; Batch: B3

1 Introduction

This assignment implements a simple CUDA program in two stages: an unparallelized, host-only version, and a version that adds a parallel GPU kernel. Both were compiled with `nvcc -arch=sm_75` and executed on a Google Colab T4 GPU runtime.

2 Part 1: Host-Only Program

This part contains only host code, no `__global__` kernel or device code is used. It is compiled with `nvcc` but runs entirely on the CPU, and serves as a baseline before introducing a GPU kernel.

```
#include <stdio.h>

int main() {
    printf("Hello World\n");
    return 0;
}
```

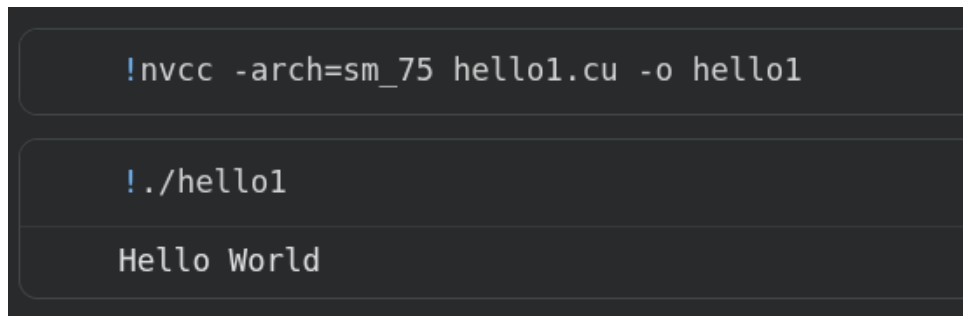


Figure 1: Output of the host-only `hello1.cu` program.

3 Part 2: Host and Device Code

This part adds a `__global__` kernel, `mykernel`, launched with configuration `<<1, 5>>`, i.e. one block of five threads. The host prints its own message first, then launches the kernel; `cudaDeviceSynchronize()` blocks the host until all five GPU threads finish, since kernel launches are asynchronous by default.

```
#include <stdio.h>

__global__ void mykernel(void) {
    printf("Hello World from GPU thread %d!\n", threadIdx.x);
}

int main() {
    printf("Hello World\n");
    mykernel<<<1, 5>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

```
!nvcc -arch=sm_75 hello2.cu -o hello2

!./hello2

Hello World
Hello World from GPU thread 0!
Hello World from GPU thread 1!
Hello World from GPU thread 2!
Hello World from GPU thread 3!
Hello World from GPU thread 4!
```

Figure 2: Output of hello2.cu, showing the host message followed by all five GPU threads.

4 Discussion of Components

- `__global__`: qualifier that marks `mykernel` as a kernel, callable from the host and executed on the device.
- `<<1, 5>>`: execution configuration specifying the kernel is launched with 1 block containing 5 threads.
- `threadIdx.x`: built-in variable giving each thread its own index within its block, used here so each thread prints a distinct message.
- `cudaDeviceSynchronize()`: blocks host execution until all previously issued device work completes; without it, `main` could return before the GPU threads finish printing.
- `nvcc -arch=sm_75`: targets compilation at compute capability 7.5, the Turing-class T4 GPU identified in Assignment 1.

5 Conclusion

The host-only program confirmed a working compilation and execution baseline, and the second program confirmed that host and device code execute together correctly, with five parallel GPU threads each producing independent output alongside the host's own print statement.